DÉMO 07 · CHANGEMENTS ENTRE LES ÉTAPES

Formulaire

Chaque étape comparée à la précédente : ce qui est ajouté et ce qui est retiré.

`+ ligne ajoutée` `– ligne retirée` `@@ position dans le fichier`

1. **Étape 1 → 2** · etape-2-zod (3 fichiers)
2. **Étape 2 → 3** · etape-3-react-hook-form (3 fichiers)

Étape 1 → 2 etape-1-usestate → etape-2-zod

Validation avec Zod seul (safeParse), sans librairie de formulaire

COMPARÉ **src/components/Form.tsx → src/components/FormOnlyZod.tsx** +31 −45

```
@@ -1,29 +1,30 @@
  import { useState } from "react";
+ import * as z from "zod";
+
+ const formSchema = z.object({
+   email: z.string().email("Email invalide"),
+   password: z
+     .string()
+     .min(1, "Mot de passe obligatoire")
+     .min(6, "Min 6 caractères"),
+ });

  function Form() {
-   const [email, setEmail] = useState("");
-   const [password, setPassword] = useState("");
    const [isError, setIsError] = useState(false);
    const [isConnected, setIsConnected] = useState(false);

-   const emailChange = (event) => {
-     const { value } = event.target;
-     const isEmailValid = value.includes("@");
-     if (isEmailValid) {
-       setEmail(value);
-     }
-   };
-
-   const passwordChange = (event) => {
-     const { value } = event.target;
-     if (value.length > 0) {
-       setPassword(value);
-     }
-   };
-
    const handleSubmit = (event) => {
      event.preventDefault();
-     if (email && password) {
+
+     // récupérer les valeurs depuis le formulaire
+     const formData = new FormData(event.currentTarget);
+
+     const email = String(formData.get("email") ?? "");
+     const password = String(formData.get("password") ?? "");
+
+     // validation Zod sans try/catch
```

```
      +     const result = formSchema.safeParse({ email, password });
      +     if (result.success) {
27    28        setIsConnected(true);
28    29      } else {
29    30        setIsError(true);
            @@ -34,34 +35,30 @@
34    35      <div className="container">
35    36        <h1>Login Component</h1>
36    37        <h2>Enter your login credentials</h2>
      38    +
37    39        {isConnected ? (
38          -       <div className="success">Welcome to private page</div>
      40    +       <div className="success">Welcome to private page</div>
39    41        ) : (
40          -       <form
41          -         name="login-form"
42          -         onSubmit={(e) => {
43          -           handleSubmit(e);
44          -         }}
45          -       >
      42    +       <form name="login-form" onSubmit={handleSubmit}>
46    43            <div className="form">
47    44              <div className="form-group">
48    45                <label>Email</label>
49          -             <input type="text" onChange={(e) => emailChange(e)} />
      46    +             {/* name obligatoire pour FormData */}
      47    +             <input name="email" type="text" />
50    48              </div>
51    49
52    50              <div className="form-group">
53    51                <label>Password</label>
54          -             <input
55          -               type="password"
56          -               onChange={(e) => {
57          -                 passwordChange(e);
58          -               }}
59          -             />
      52    +             {/* name obligatoire pour FormData */}
      53    +             <input name="password" type="password" />
60    54              </div>
      55    +
61    56              {isError && (
62          -             <p className="error">Please enter valid email and password.</p>
      57    +             <p className="error">
      58    +               {"Please enter valid email and password."}
      59    +             </p>
63    60              )}
64          -           {isConnected && <p className="success">Welcome to private page</p>}
      61    +
65    62              <button className="btn" type="submit">
66    63                Login
67    64              </button>
            @@ -73,14 +70,3 @@
73    70    }
74    71
75    72    export default Form;
76          -
77          - /*
78          - // Email (version correcte)
79          - const emailRegex = /^[^\s@]+@[^\s@]+\.[^\s@]+$/;
80          -
81          - // Password: au moins 6 caractères
82          - const passwordRegex = /^.{6,}$/;
83          -
```

```
84  - const isEmailValid = emailRegex.test(email);
85  - const isPasswordValid = passwordRegex.test(password);
86  - */
```

MODIFIÉ **src/App.jsx** +2 −1

```
@@ -1,9 +1,10 @@
1  1    import "./App.css";
2  2    import Form from "./components/Form";
3  3    import FormFinal from "./components/FormFinal";
   4  + import FormOnlyZod from "./components/FormOnlyZod";
4  5
5  6    function App() {
6     -   return <Form />;
   7  +   return <FormOnlyZod />;
7  8    }
8  9
9  10   export default App;
```

UTILISÉ **src/components/FormOnlyZod.tsx** +72 −0

```
@@ -1,0 +1,72 @@
1  + import { useState } from "react";
2  + import * as z from "zod";
3  +
4  + const formSchema = z.object({
5  +   email: z.string().email("Email invalide"),
6  +   password: z
7  +     .string()
8  +     .min(1, "Mot de passe obligatoire")
9  +     .min(6, "Min 6 caractères"),
10 + });
11 +
12 + function Form() {
13 +   const [isError, setIsError] = useState(false);
14 +   const [isConnected, setIsConnected] = useState(false);
15 +
16 +   const handleSubmit = (event) => {
17 +     event.preventDefault();
18 +
19 +     // récupérer les valeurs depuis le formulaire
20 +     const formData = new FormData(event.currentTarget);
21 +
22 +     const email = String(formData.get("email") ?? "");
23 +     const password = String(formData.get("password") ?? "");
24 +
25 +     // validation Zod sans try/catch
26 +     const result = formSchema.safeParse({ email, password });
27 +     if (result.success) {
28 +       setIsConnected(true);
29 +     } else {
30 +       setIsError(true);
31 +     }
32 +   };
33 +
34 +   return (
35 +     <div className="container">
36 +       <h1>Login Component</h1>
37 +       <h2>Enter your login credentials</h2>
38 +
39 +       {isConnected ? (
40 +         <div className="success">Welcome to private page</div>
41 +       ) : (
42 +         <form name="login-form" onSubmit={handleSubmit}>
43 +           <div className="form">
```

```
+             <div className="form-group">
+               <label>Email</label>
+               {/* name obligatoire pour FormData */}
+               <input name="email" type="text" />
+             </div>
+
+             <div className="form-group">
+               <label>Password</label>
+               {/* name obligatoire pour FormData */}
+               <input name="password" type="password" />
+             </div>
+
+             {isError && (
+               <p className="error">
+                 {"Please enter valid email and password."}
+               </p>
+             )}
+
+             <button className="btn" type="submit">
+               Login
+             </button>
+           </div>
+         </form>
+       )}
+     </div>
+   );
+ }
+
+ export default Form;
```

Étape 2 → 3 etape-2-zod → etape-3-react-hook-form

Version finale : React Hook Form + Zod + zodResolver (register, handleSubmit, errors)

COMPARÉ `src/components/FormOnlyZod.tsx → src/components/FormFinal.tsx` +69 −31

```
@@ -1,34 +1,41 @@
- import { useState } from "react";
- import * as z from "zod";
+ import { useForm } from "react-hook-form";
+ import { z } from "zod";
+ import { zodResolver } from "@hookform/resolvers/zod";

- const formSchema = z.object({
+ const schema = z.object({
    email: z.string().email("Email invalide"),
    password: z
      .string()
      .min(1, "Mot de passe obligatoire")
      .min(6, "Min 6 caractères"),
+   // Champ supplémentaire "Experience"
+   experience: z
+     .number()
+     .int("Nombre entier uniquement")
+     .min(0, "Min = 0")
+     .max(10, "Max = 10"),
+   // Checkbox "Terms & Conditions"
+   terms: z
+     .boolean()
+     .refine(
+       (v) => v === true,
+       "Vous devez accepter les Conditions d'utilisation"
+     ),
  });

  function Form() {
-   const [isError, setIsError] = useState(false);
-   const [isConnected, setIsConnected] = useState(false);
+   const {
+     register,
+     handleSubmit,
+     formState: { errors, isSubmitSuccessful, isValid },
+   } = useForm({
+     resolver: zodResolver(schema),
+     defaultValues: { email: "", password: "", experience: 0, terms: false },
+     mode: "onSubmit",
+   });

-   const handleSubmit = (event) => {
-     event.preventDefault();
-
-     // récupérer les valeurs depuis le formulaire
-     const formData = new FormData(event.currentTarget);
-
-     const email = String(formData.get("email") ?? "");
-     const password = String(formData.get("password") ?? "");
-
-     // validation Zod sans try/catch
-     const result = formSchema.safeParse({ email, password });
-     if (result.success) {
-       setIsConnected(true);
-     } else {
-       setIsError(true);
-     }
```

```
     37 +   const onSubmit = (data: any) => {
     38 +     console.log("Send to server:", data); // ici tu fais ton appel API
32   39     };
33   40
34   41     return (
          @@ -36,28 +43,59 @@
36   43         <h1>Login Component</h1>
37   44         <h2>Enter your login credentials</h2>
38   45
39      -       {isConnected ? (
     46 +       {isSubmitSuccessful ? (
40   47           <div className="success">Welcome to private page</div>
41   48         ) : (
42      -         <form name="login-form" onSubmit={handleSubmit}>
     49 +         <form name="login-form" onSubmit={handleSubmit(onSubmit)}>
43   50             <div className="form">
44   51               <div className="form-group">
45   52                 <label>Email</label>
46      -                {/* name obligatoire pour FormData */}
47      -                <input name="email" type="text" />
     53 +                <input type="text" {...register("email")} />
     54 +                {errors.email && (
     55 +                  <p className="error">{errors.email.message as string}</p>
     56 +                )}
48   57               </div>
49   58
50   59               <div className="form-group">
51   60                 <label>Password</label>
52      -                {/* name obligatoire pour FormData */}
53      -                <input name="password" type="password" />
     61 +                <input type="password" {...register("password")} />
     62 +                {errors.password && (
     63 +                  <p className="error">{errors.password.message as string}</p>
     64 +                )}
54   65               </div>
55   66
56      -             {isError && (
57      -               <p className="error">
58      -                 {"Please enter valid email and password."}
59      -               </p>
     67 +             <div className="form-group">
     68 +               <label>Experience</label>
     69 +               <input
     70 +                 type="number" // Changé de "text" à "number"
     71 +                 {...register("experience", { valueAsNumber: true })} // Ajouté valueAsNumbe
        r
     72 +               />
     73 +               {errors.experience && (
     74 +                 <p className="error">{errors.experience.message as string}</p>
     75 +               )}
     76 +             </div>
     77 +
     78 +             {/* Terms & Conditions */}
     79 +             <div className="form-group">
     80 +               <label className="checkbox">
     81 +                 <input type="checkbox" {...register("terms")} />
     82 +                 <span>
     83 +                   J'accepte les{" "}
     84 +                   <a href="#" target="_blank" rel="noreferrer">
     85 +                     Conditions d'utilisation
     86 +                   </a>
     87 +                 </span>
     88 +               </label>
     89 +               {errors.terms && (
     90 +                 <p className="error">{errors.terms.message as string}</p>
```

```
                )}
              </div>

              {Object.keys(errors).length > 0 && (
                <p className="error">Veuillez corriger les champs invalides.</p>
              )}

              {isValid && <p className="success">Form valid (ready to send)</p>}

              <button className="btn" type="submit">
                Login
```

MODIFIÉ `src/App.jsx` +1 −2

```
@@ -1,10 +1,9 @@
 import "./App.css";
 import Form from "./components/Form";
 import FormFinal from "./components/FormFinal";
-import FormOnlyZod from "./components/FormOnlyZod";

 function App() {
-  return <FormOnlyZod />;
+  return <FormFinal />;
 }

 export default App;
```

UTILISÉ `src/components/FormFinal.tsx` +110 −0

```
@@ -1,0 +1,110 @@
+import { useForm } from "react-hook-form";
+import { z } from "zod";
+import { zodResolver } from "@hookform/resolvers/zod";
+
+const schema = z.object({
+  email: z.string().email("Email invalide"),
+  password: z
+    .string()
+    .min(1, "Mot de passe obligatoire")
+    .min(6, "Min 6 caractères"),
+  // Champ supplémentaire "Experience"
+  experience: z
+    .number()
+    .int("Nombre entier uniquement")
+    .min(0, "Min = 0")
+    .max(10, "Max = 10"),
+  // Checkbox "Terms & Conditions"
+  terms: z
+    .boolean()
+    .refine(
+      (v) => v === true,
+      "Vous devez accepter les Conditions d'utilisation"
+    ),
+});
+
+function Form() {
+  const {
+    register,
+    handleSubmit,
+    formState: { errors, isSubmitSuccessful, isValid },
+  } = useForm({
+    resolver: zodResolver(schema),
+    defaultValues: { email: "", password: "", experience: 0, terms: false },
+    mode: "onSubmit",
+  });
```

```
+
+  const onSubmit = (data: any) => {
+    console.log("Send to server:", data); // ici tu fais ton appel API
+  };
+
+  return (
+    <div className="container">
+      <h1>Login Component</h1>
+      <h2>Enter your login credentials</h2>
+
+      {isSubmitSuccessful ? (
+        <div className="success">Welcome to private page</div>
+      ) : (
+        <form name="login-form" onSubmit={handleSubmit(onSubmit)}>
+          <div className="form">
+            <div className="form-group">
+              <label>Email</label>
+              <input type="text" {...register("email")} />
+              {errors.email && (
+                <p className="error">{errors.email.message as string}</p>
+              )}
+            </div>
+
+            <div className="form-group">
+              <label>Password</label>
+              <input type="password" {...register("password")} />
+              {errors.password && (
+                <p className="error">{errors.password.message as string}</p>
+              )}
+            </div>
+
+            <div className="form-group">
+              <label>Experience</label>
+              <input
+                type="number" // Changé de "text" à "number"
+                {...register("experience", { valueAsNumber: true })} // Ajouté valueAsNumbe
r
+              />
+              {errors.experience && (
+                <p className="error">{errors.experience.message as string}</p>
+              )}
+            </div>
+
+            {/* Terms & Conditions */}
+            <div className="form-group">
+              <label className="checkbox">
+                <input type="checkbox" {...register("terms")} />
+                <span>
+                  J’accepte les{" "}
+                  <a href="#" target="_blank" rel="noreferrer">
+                    Conditions d’utilisation
+                  </a>
+                </span>
+              </label>
+              {errors.terms && (
+                <p className="error">{errors.terms.message as string}</p>
+              )}
+            </div>
+
+            {Object.keys(errors).length > 0 && (
+              <p className="error">Veuillez corriger les champs invalides.</p>
+            )}
+
+            {isValid && <p className="success">Form valid (ready to send)</p>}
+
```

```
100 +             <button className="btn" type="submit">
101 +               Login
102 +             </button>
103 +           </div>
104 +         </form>
105 +       )}
106 +     </div>
107 +   );
108 + }
109 +
110 + export default Form;
```